

DS2 option info MPSI

Algorithme polynomial pour le problème 2-SAT

Ce sujet vise à pour objectif d'arriver à un algorithme polynomial pour résoudre le problème 2-SAT. Même si c'est l'objet d'étude du sujet, aucune connaissance n'est requise sur le problème SAT ici : on va simplement manipuler des formules logiques et des graphes.

I) Formules en 2-CNF

Définition 1 [Formule sous forme 2-FNC]

Une formule logique est dite en 2-FNC si elle est de la forme :

$$\varphi = \bigwedge_{i=1}^n p_i \vee q_i$$

où p_i et q_i sont des littéraux, c'est-à-dire soit des variables x , soit des négations de variables $\neg x$.

Les $p_i \vee q_i$ sont appelés les clauses.

Exemple 2

La formule $\varphi = (a \vee b) \wedge (\neg a \vee \neg b) \wedge (c \vee \neg c)$ est en 2-FNC.

Les formules en 2-FNC seront représentées en OCaml par le type suivant :

```
type littéral = Var of int | Not of int;;  
type formule = (littéral * littéral) list;;
```

On va considérer que les variables d'une formule sont numérotées par des entiers *consécutifs* entre 0 et $n - 1$, où n est le nombre de variables distinctes de la formule.

Par exemple, en numérotant a, b et c par 0, 1 et 2, la formule φ est représentée par la liste :

```
let phi: formule = [(Var 0, Var 1); (Not 0, Not 1); (Var 2, Not 2)];;
```

On va s'intéresser à la résolution de formules en 2-FNC, c'est à dire de trouver une valuation telles que la formule soit vraie, si c'est possible.

On dit qu'une formule est *satisfiable* si une telle valuation existe.

Q1) La formule φ de l'exemple 2 est-elle satisfiable ? Si oui, donnez une valuation des variables telle qu'elle s'évalue à Vrai.

Q2) Toutes les formules peuvent-elles être mises sous la forme 2-FNC ? Vous pouvez par exemple étudier la formule $a \vee b \vee c$ pour répondre.

Q3) Écrivez une fonction `neg : littéral -> littéral` qui renvoie la négation d'un littéral (c'est à dire qu'elle transforme a en $\neg a$ et $\neg a$ en a).

Q4) Écrivez une fonction `nb_littéraux : formule -> int` qui calcule le nombre de littéraux dans une formule (en 2-FNC toujours, comme sur tout le reste du sujet).

Q5.) Écrivez une fonction `nb_variables : formule -> int` qui calcule le nombre de variables différentes dans une formule.

*Ne cherchez pas trop loin ! En particulier, les variables sont des entiers **consécutifs**...*

Pour représenter une valuation, on va utiliser un tableau. Comme on a déjà numéroté les variables entre 0 et $n - 1$, on peut utiliser cette même numérotation pour la valuation, et placer à l'emplacement i du tableau la valeur de la variable i .

```
type valuation = bool array;;
```

Ainsi, pour la formule φ , la valuation :

a	Vrai
b	Vrai
c	Faux

s'écrit en OCaml :

```
let val: valuation = [|true; true; false|]
```

Q6.) Écrivez une fonction `eval: formule -> valuation -> bool` qui évalue une formule avec une valuation.

On supposera que la valuation contient bien toutes les variables de la formule.

II) Graphe d'implication

Pour résoudre de telles formules, on va utiliser une construction appelée *graphe d'implication*.

II.1) Définition

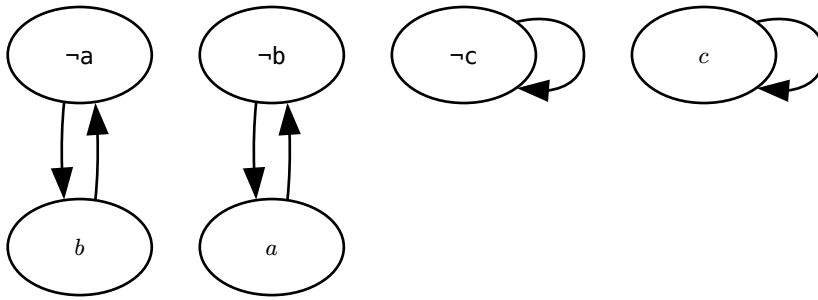
Définition 3 [Graphe d'implication]

Pour une formule φ en 2-FNC, on appelle **graphe d'implication** le graphe orienté $G = (S, A)$ dont :

- les sommets S sont les littéraux de φ (c'est à dire, toutes les variables de φ et leurs négations)
- pour chaque clause $p \vee q$, on place les arêtes $\neg p \rightarrow q$ et $\neg q \rightarrow p$

nb : en fait, on peut voir la clause $p \vee q$ comme l'implication "non p implique q ", et donc les arêtes de graphe sont des implications.

Par exemple, le graphe correspondant à la formule φ est :



Q7.) Représentez le graphe d'implication correspondant à la formule

$$\psi = (a \vee a) \wedge (\neg a \vee b) \wedge (\neg b \vee c) \wedge (\neg c \vee d) \wedge (\neg d \vee \neg a)$$

Q8.) Que peut-on en déduire sur la formule si, pour une variable x , le graphe contient une arrête $x \rightarrow \neg x$ et une arrête $\neg x \rightarrow x$? La formule est-elle alors satisfiable ?

Q9.) Montrer que, pour x une variable, si le graphe contient un chemin entre x et $\neg x$ et un chemin entre $\neg x$ et x , alors la formule n'est pas satisfiable.

On admet dans un premier temps que la réciproque est également vraie.

II.2) Implémentation en OCaml

On choisi de représenter les graphes par liste d'adjacence.

Les sommets correspondant aux variables sont numérotés de 0 à $n - 1$, et ceux correspondant à leurs négations de n à $2n - 1$.

On peut alors utiliser le type suivant pour représenter les graphes :

```
type graphe = int list array
```

où l'élément d'indice i du tableau contiens la listes des voisins de i , c'est à dire les sommets v tels que $i \rightarrow v$.

Q10.) Écrivez une fonction `index: littéral -> int` qui renvoie l'indice du sommet correspondant à un littéral dans le graphe.

Q11.) Écrivez une fonction `f2g : formule -> graphe` qui calcule et renvoie le graphe d'implication d'une formule.

Définition 4 [Composantes fortement connexes]

Pour $G = (S, A)$ un graphe orienté, on appelle *composante fortement connexe* de G tout sous-ensembles des sommets $C \in S$ tel que, pour tout $p, q \in C$, il existe un chemin de p à q et de q à p .

Q12.) Montrez que la condition énoncée à la question 9 est équivalente à “pour toute variable x , x et $\neg x$ ne sont pas dans la même composante fortement connexe.

On va donc calculer les composantes fortement connexes de notre graphe.

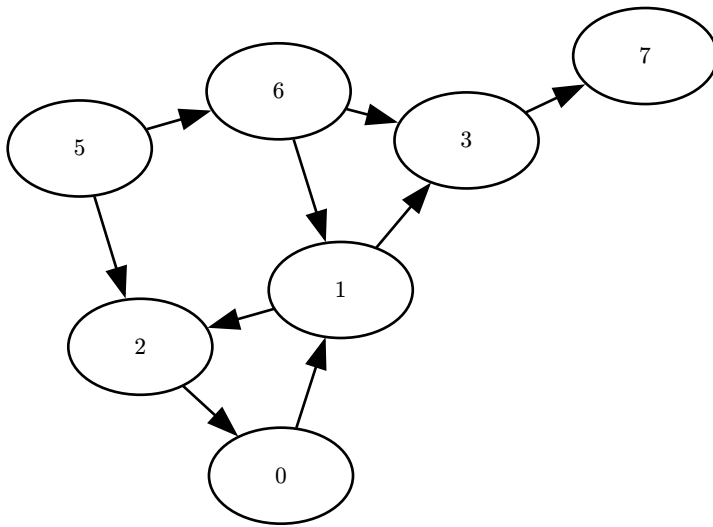
On donne la fonction suivante :

```

let rec parcours (g: graphe) (s: int) (visite: bool array): int list =
  if visite.(s) then [] else
  let rec parcours_voisins (l: int list): int list = match l with
    | [] -> []
    | v::q -> (parcours g v visite) @ (parcours_voisins q)
  in
  visite.(s) <- true;
  s::(parcours_voisins g.(s))

```

Q13.) Que fait cette fonction ? Donnez son résultat sur le graphe suivant, en partant de 0, avec le tableau visite rempli de false au début.



Q14.) * Cette fonction ne parcourt pas le graphe dans son intégralité. Écrivez une fonction `parcours_total: graphe -> int list` qui parcourt l'intégralité du graphe avec la fonction `parcours` (en l'appelant plusieurs fois).

La fonction `parcours_total` doit renvoyer la concaténation des résultats de tous les appels à `parcours`.

On va utiliser un algorithme classique (enfin, "classique", c'est relatif) pour calculer les composantes fortement connexes de notre graphe :

Algorithme 5 [de Kosaraju]

- Appeler la fonction `parcours_total` et noter l'ordre renvoyé ω , c'est à dire l'ordre de remontée du parcours.
- Transposer le graphe G en G^T (c'est à dire inverser toutes ses arrêtes).
- Parcourir G^T , en suivant l'ordre indiqué par ω , c'est-à-dire que, lorsqu'un appel à `parcours` est terminé, on reprend depuis le premier sommet de ω non visité.

Chaque appel à `parcours` renvoie alors une composante connexe.

Q15.) Écrivez une fonction `transpose: graphe -> graphe` qui renvoie la transposée d'un graphe, c'est-à-dire que toutes ses arrêtes sont inversées.

Q16.) * Écrivez une fonction `kosaraju: graphe -> int list list` qui prend en argument un graphe et renvoie la liste de ses composantes connexes (donc une liste de listes de sommets).

Q17) * En combinant les différentes fonctions du sujet, écrivez une fonction satisfiable :
formule $\rightarrow$ bool qui détermine si une formule sous forme 2-FNC est satisfiable. Quelle-est sa complexité ?

Q18) Comment peut-on attribuer des valeurs aux variables qui satisfont la formule à l'aide du graphe d'implication ?